

Linear Regression & Regularization

=> Supervised Learning

Supervised learning :

- **Regression** → we are trying to predict **continuous numerical values**
- **Classification** → we are trying to predict **discrete categories/labels**

=> Simple Linear Regression

Prediction:

$$y = wx + b$$

- w → weight / slope (parameter)
- b → bias / intercept(parameter)

=> Multiple Linear Regression

When we have more than one feature

$$y = w_1 * x_1 + w_2 * x_2 + \dots + b$$

=>Cost Function

The cost function measures the distance between the prediction and the real value (how wrong the model's predictions are)

$$J(w, b) = (1 / 2m) * \sum (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- Why squared?→ because the raw difference $(h(x) - y)$ could be negative so squaring turns it into a proper (always positive)
"squared error function"
- Why divide by 'm'? -> to compute the average error over all training examples
Why divide by 2? ->it simplifies the derivative

The Cost Function in Linear Regression is based on MSE

$$= (1/n) \sum (y - \hat{y})^2$$

=> Gradient Descent

Idea:

- Start with some initial w, b

- Keep changing w , b until we settle near a minimum of the cost function

Note: In linear regression, the cost function is convex "bowl-shaped" so it doesn't matter too much what the initial values of w and b are

Update Rule

```
repeat until convergence {  
     $w = w - \alpha * (\partial/\partial w) J(w, b)$   
     $b = b - \alpha * (\partial/\partial b) J(w, b)$   
}
```

- α (alpha) -> **learning rate**: controls how big each step is
- $\partial J/\partial w$, $\partial J/\partial b$ -> derivatives (gradient): tell us the direction to move

Effect of Learning Rate (α)

α value	Effect
Too small	Gradient descent takes tiny steps -> converges very slowly
Too large	May overshoot the minimum, oscillate back and forth, and never converge

$$w = w - \alpha \frac{d}{dw} J(w)$$

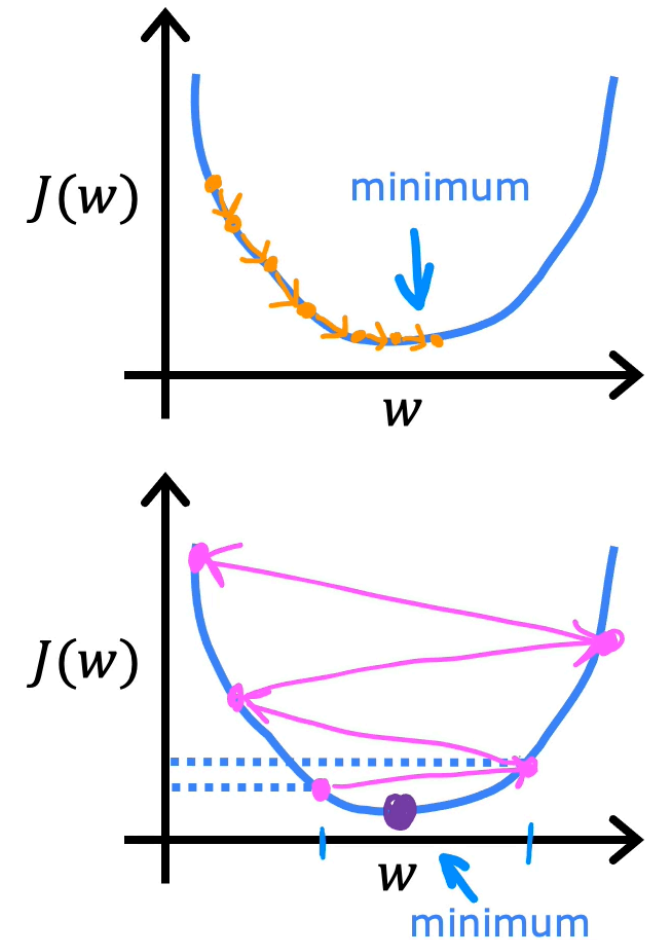
If α is too small...

Gradient descent may be slow.

If α is too large...

Gradient descent may:

- Overshoot, never reach minimum



=>Performance Metrics

-> R^2 and Adjusted R^2

- R^2 always increases (or stays the same) as you add more features (even irrelevant ones)
- **Adjusted R^2** penalizes adding useless features, giving a fairer measure of model performance when comparing models with different numbers of features.

- $R^2 = 1 \rightarrow$ Perfect prediction
- $R^2 = 0 \rightarrow$ Model explains none of the variance

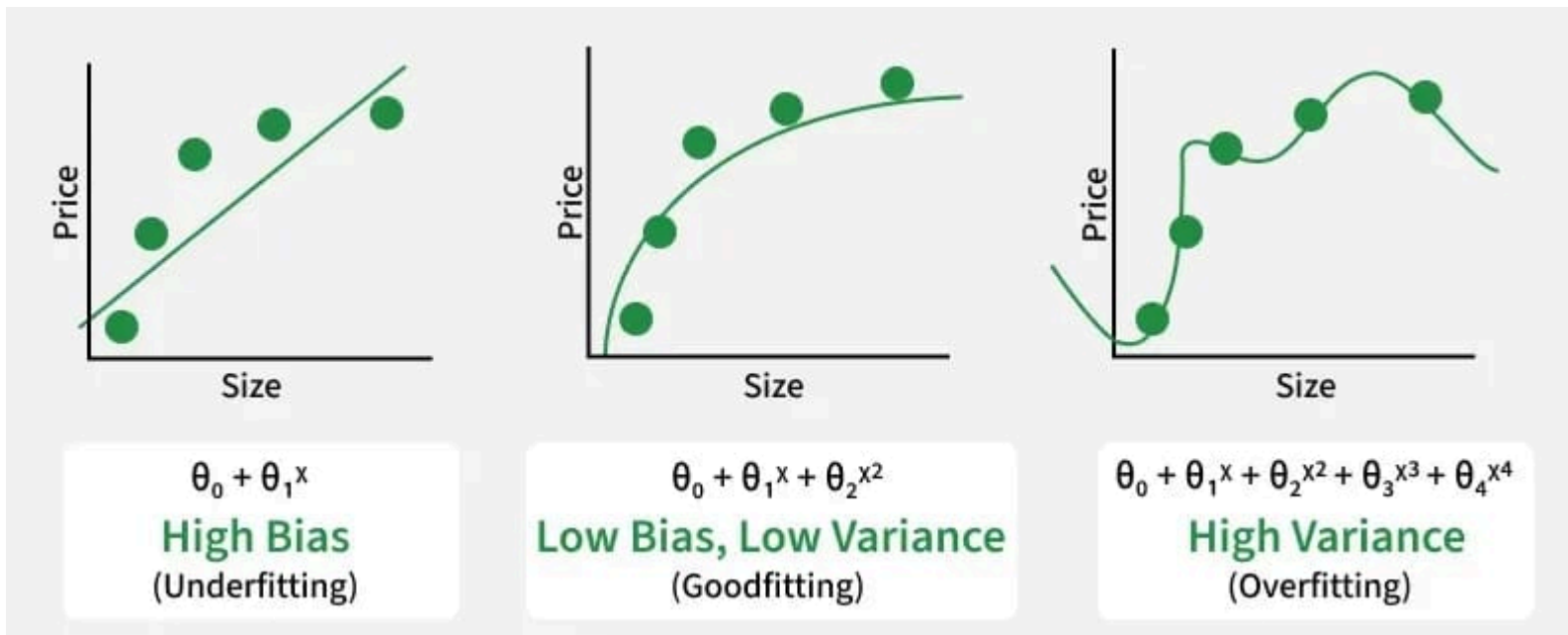
=>Overfitting vs Underfitting

->Overfitting:

- model performs well on training data (low bias)
- fails to perform well on test data (high variance)
- model has essentially "memorized" the training data instead of learning

->Underfitting:

- model accuracy is bad on training data
- and bad on test data
- ->high bias (too simple to capture the pattern)



=>Regularization

Regularization adds a penalty term to the cost function to **prevent overfitting** by discouraging overly large weights.

->Ridge Regression (L2 Regularization)

Cost function:

$$J = \sum (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda * (\text{slope})^2$$

Purpose:

Prevents overfitting by shrinking the weights (slopes) but doesn't force them to be exactly zero.

Why don't we regularize the Bias?

as it doesn't control the model complexity

->Lasso Regression (L1 Regularization)

Cost function:

$$J = \sum (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda * |slope|$$

Purpose:

1. Prevents overfitting

2. Feature selection ->

can shrink some weights all the way to zero

effectively removing unimportant features from the model